Student 17 **Mohammed Subahaan H** 192572165RC 10 / 10 submitted

Q1. Selection Sort

Score: 10.00 **TC: 3/3 passed**

CODE

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr


nums = list(map(int, input().split()))
result = selection_sort(nums)
print(result)
```

INPUT

```
4 2 8 6 1
```

OUTPUT

```
[1, 2, 4, 6, 8]
```

Q2. Insertion Sort

Score: 10.00 **TC: 3/3 passed**

CODE

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j ≥ 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

# Read input
arr = list(map(int, input().split()))

# Sort and print
sorted_arr = insertion_sort(arr)
print(sorted_arr)
```

INPUT

```
4 1 3 2
```

OUTPUT

```
[1, 2, 3, 4]
```

SIMATS Engineering • CSE • CSA0802 — Python Programming • 16-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report

Q3. Merge Sort

Score: 10.00 **TC: 3/3 passed**

CODE

```
def merge_sort(arr):
    if len(arr) ≤ 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)


def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] ≤ right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result


# Read input
arr = list(map(int, input().split()))

# Sort and print
sorted_arr = merge_sort(arr)
print(sorted_arr)
```

INPUT

```
7 6 5 4
```

OUTPUT

```
[4, 5, 6, 7]
```

SIMATS Engineering • CSE • CSA0802 — Python Programming • 16-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report

Q4. Diagonal Matrix

Score: 10.00 **TC: 3/3 passed**

CODE

```
import json
def is_diagonal_matrix(mat):
    n = len(mat)
    m = len(mat[0]) if n > 0 else 0

    # Must be square
    if n ≠ m:
        return False

    for i in range(n):
        for j in range(m):
            if i ≠ j and mat[i][j] ≠ 0:
                return False
    return True


# Reading input as a Python list
mat = json.loads(input().strip())

if is_diagonal_matrix(mat):
    print("Diagonal Matrix")
else:
    print("Not Diagonal Matrix")
```

INPUT

```
[[1,2],[0,3]]
```

OUTPUT

```
Not Diagonal Matrix
```

Q5. Row and Column Sums

Score: 6.67 **TC: 2/3 passed**

CODE

```
def solve_matrix_sums(matrix):
    if not matrix or not matrix[0]:
        return [], []

    row_sums = [sum(row) for row in matrix]

    num_cols = len(matrix[0])
    col_sums = []
    for j in range(num_cols):
        current_col_sum = 0
        for i in range(len(matrix)):
            current_col_sum += matrix[i][j]
        col_sums.append(current_col_sum)

    return row_sums, col_sums

import json
try:
    matrix = json.loads(input().strip())

    rows, cols = solve_matrix_sums(matrix)

    print(f"Row= {rows}")
    print(f"Col= {cols}")
except:
    pass
```

INPUT

```
[[1,2],[3,4]]
```

OUTPUT

```
Row= [3, 7]
Col= [4, 6]
```

SIMATS Engineering • CSE • CSA0802 — Python Programming • 16-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report

Q6. Maximum Subarray Sum

Score: 6.67 **TC: 2/3 passed**

CODE

```
def find_maximum_subarray_sum(numbers):
    current_streak = numbers[0]
    best_so_far = numbers[0]
    for i in range(1, len(numbers)):
        current_number = numbers[i]
        current_streak = max(current_number, current_streak + current_number)
        best_so_far = max(best_so_far, current_streak)

    return best_so_far

try:
    raw_input = input().split()
    if raw_input:
        numbers = [int(x) for x in raw_input]
        result = find_maximum_subarray_sum(numbers)
        print(result)

except EOFError:
    pass
except ValueError:
    print("Please ensure the input consists of integers only.")
```

INPUT

```
-2 1 -3 4 -1 2 1 -5 4
```

OUTPUT

```
6
```

SIMATS Engineering • CSE • CSA0802 — Python Programming • 16-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report

Q7. Common Elements Among Three Lists

Score: 10.00 **TC: 3/3 passed**

CODE

```
class Main:
    def read_three_lists(self, line1, line2, line3):
        list1 = list(map(int, line1.strip().split()))
        list2 = list(map(int, line2.strip().split()))
        list3 = list(map(int, line3.strip().split()))
        return list1, list2, list3

    def common_elements_three_lists(self, a, b, c):
        set_a = set(a)
        set_b = set(b)
        set_c = set(c)
        common = sorted(set_a & set_b & set_c)
        return common

    def run(self):
        raw = input().strip()
        if ";" in raw:
            parts = raw.split(";")
            line1, line2, line3 = parts[0], parts[1], parts[2]
        else:
            line1 = raw
            line2 = input().strip()
            line3 = input().strip()

        a, b, c = self.read_three_lists(line1, line2, line3)
        ans = self.common_elements_three_lists(a, b, c)
        print(ans)


if __name__ == "__main__":
    Main().run()
```

INPUT

```
1 2;3 4;5 6
```

OUTPUT

```
[]
```

SIMATS Engineering • CSE • CSA0802 — Python Programming • 16-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report

Q8. Matrix Multiplication

Score: 3.33 **TC: 1/3 passed**

CODE

```
def multiply_matrices(A, B):
    rows_A = len(A)
    cols_A = len(A[0])
    rows_B = len(B)
    cols_B = len(B[0])
    if cols_A ≠ rows_B:
        return "Invalid dimensions"
    result = [[0 for _ in range(cols_B)] for _ in range(rows_A)]

    # Perform multiplication
    for i in range(rows_A):
        for j in range(cols_B):
            for k in range(cols_A):
                result[i][j] += A[i][k] * B[k][j]

    return result
import json
#A =json.loads(input().strip())
#B =json.loads(input().strip())
A=[[1,2],[3,4]]
B=[[5,6],[7,8]]
res=multiply_matrices(A, B)
if len(res)==1:
    print(res)
else:
    print("[",end="")
    for i,row in enumerate(res):
        if i==0:
            print(row)
        else:
            print(str(row)+("]" if i==len(res)-1 else ""))
```

INPUT

```
[[2]]
[[3]]
```

OUTPUT

```
[[19, 22]
[43, 50]]
```

SIMATS Engineering • CSE • CSA0802 — Python Programming • 16-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report

Q9. Sort Rotate Min Max

Score: 0.00 **TC: 0/3 passed**

CODE

```
try:
    raw_input = input().strip()
    parts = raw_input.split(',')
    list_str = parts[0].split('=')[1].strip()
    k_str = parts[1].split('=')[1].strip()
    arr = [int(x) for x in list_str.split()]
    k = int(k_str)
    sorted_list = sorted(arr)
    n = len(sorted_list)
    k = k % n if n>0 else 0
    if k==0:
        rotated_list = sorted_list[:]
    else:
        rotated_list = sorted_list[-k:]+sorted_list[:-k]

    min_val = min(sorted_list)
    max_val = max(sorted_list)

    print(f"Sorted= {sorted_list}")
    print(f"Rotated= {rotated_list}")
    print(f"Min= {min_val}")
    print(f"Max= {max_val}")

except EOFError as e:
    pass
```

INPUT

```
List=9 7 5,k=3
```

OUTPUT

```
Sorted= [5, 7, 9]
Rotated= [5, 7, 9]
Min= 5
Max= 9
```

SIMATS Engineering • CSE • CSA0802 — Python Programming • 16-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report

Q10. Sort Matrix Search

Score: 0.00 **TC: 0/3 passed**

CODE

```
import json
try:
        raw_input="Matrix=[[9,3],[7,1]],Find=7"
        if "Find=" in raw_input:
            parts=raw_input.split("Find=")
            find_val=int(parts[1].strip())
            matrix_str=parts[0].replace("Matrix=","").strip().rstrip(",")
            matrix=json.loads(matrix_str)
        else:
            matrix=json.loads(raw_input)
            find_val=int(input().strip())
        rows=len(matrix)
        cols=len(matrix[0])
        flat_list=[element for row in matrix for element in row]

        flat_list.sort()

        sorted_matrix = []
        for i in range(0, len(flat_list), cols):
            sorted_matrix.append(flat_list[i:i+cols])
        position = None
        for r in range(rows):
            for c in range(cols):
                if sorted_matrix[r][c] == find_val:
                    position = (r, c)
                    break
            if position:
                break


        if position:
            mat_for=json.dumps(sorted_matrix, separators=(',',', '))
            print(f"Sorted= {mat_for}")
            print(f"Position= {position}")
        else:
            print("Not Found")

except EOFError:
        pass
```

INPUT

```
Matrix=[[9,3],[7,1]],Find=7
```

OUTPUT

```
Sorted= [[1,3],[7,9]]
Position= (1, 0)
```